



Faculty of Computers &
Artificial Intelligence



Benha University

Driver Monitoring System (DMS)

Graduation Project

Project Team

- ♦ Ahmed Abdelaal Ahmed Abo-Hadeed.
- ♦ Ali Nabil Ali .
- ♦ idris tarek El-Sayed.
- ♦ Elsayed Sameh Elsayed.
- ♦ Ahmed Mohamed Gouda.
- ♦ Rania Yasser Mohamed.
- ♦ Shereen Ebrahiem Ebrahiem
- ♦ Eman Hamada Abdelhady .
- ♦ Khlood Elsayed Elsayed.

Supervised by : Dr / Mostafa Abdelsallam
ENG/ Sara Abdel-Mohsen

Supported by : Valeo

Abstract

It goes without saying that we extremely became dependent on means of transportation, especially cars, so safety is a must for drivers, specifically for those who drive for a long time.

When it comes to safety, it must be mentioned that in order for a driver to be fit to drive, he must be focused, driving with full concentration and alert. But first, what is the meaning of drowsiness?

Drowsiness is the feeling heavy-eyed or exhausted and being unable to keep your eyes open. The drowsiness levels differ between drivers, which means that the risk levels also differ on the road.

By reviewing other related work about this problem, we found the following techniques used: monitoring steering wheel movement as when driver doesn't have full control on the steering wheel it will lead to unusual driving pattern, tracking the acceleration and the brakes the user makes during his ride using sensors, and the last technique is Computer Vision techniques and deep learning for driver monitoring: this is done by detecting the driver's face using a camera and process the input using deep learning to output his drowsiness level based on yawning or blinking rate. After reviewing these techniques, we found the last technique the most efficient and suitable.

Our project aims to develop a system for detecting driver's drowsiness such as yawning or blinking and taking action based on driver's drowsiness level (low alert for low level ,high alert for high level and call to an emergency contact) using deep learning which will get the picture frames from the camera present in the car and will use jetson Nano as a hardware to process the model and output the result of the drowsiness and output an alert which will then lead to a safer driving for long hours without fear of falling asleep.

Driver Monitor System (DMS)

- Sending alarm actively
- Reducing traffic accident injuries
- Improving fleet operation efficiency



Abnormal behavior
detection and alarm

AI

AI deep learning

Table of Contents

CHAPTER- 01

Introduction

- 1.1 Introduction.....
- 1.2 Fact & Statistic.....
- 1.3 Motivation
- 1.4 Objective.....
- 1.5 Techniques Found for this solution
- 1.6 Advantages over other techniques
- 1.7 Target Audience

CHAPTER- 02

Proposed System

- 2.1 Limitation of our System.....
- 2.2 Future Enhancement
- 2.3 Features of our System
- 2.4 Proposed System.....
- 2.5 Conclusion
- 2.6 Software Tools.....
- 2.7 Hardware Tools.....

CHAPTER- 03

Literature Review

- 3.1 Single Board Computer Literature Review.....
- 3.2 Camera Literature Review.....
- 3.3 GPS/GSM module Literature Review.....

CHAPTER- 04

Project Scope

4.1 Features.....

4.2 Activity Diagram

4.3 Use Cases

4.4 Class diagram.....

4.5 ERD.....

4.6 Flow Chart.....

4.7 data flow diagram.....

CHAPTER- 05

Software Models

5.1 open or close eye.....

5.2 yawn or not yawn model.....

5.3 Mediapipe_eye state.....

5.4 Deploying models on our Embedded Target.....

Chapter One

1.1 Introduction

During the first chapter of this document, we are going to walk through the motives of the document. First of all, due to the huge importance of the problem we are facing, the motivation behind our solution will be presented. Furthermore, the problem, that is being faced, is going to be stated. Providing the problem's seriousness, other techniques, that are used to overcome this problem, are going to be stated and the reason why our technique has the advantage of other techniques. Last but not least, the audience targeted will be declared. According to the data information, more than 1.2 million people die every year due to road accidents and more than 20 million people experienced non-serious injuries due to road accidents [1][20]. Various studies have suggested that around 20% of all road accidents are fatigue-related, up to 50% on certain roads. For this purpose, the development of driver monitoring system has become very important. This system mostly identifies the driver's distraction level by the interval of head movement and head swing. We present a system for analyzing driver visual attention. The method is based on the estimation of the overall figures to accurately detect a person's head and facial features. The system classifies rotation in all viewing directions. In addition; it is able to detect the eye closure as well as head movement. The system accurately detects the drowsiness and distraction by using the accurate measure of algorithms and it can be used for more advanced driver visual attention monitoring. The system has been tested with different driving habits and with different users. In order to avoid these destructive accidents, the status of drowsiness of the driver should be monitored. The behavioural measures has been used extensively for monitoring drowsiness like eye closure, yawning, head pose and head movement is monitored through a particular camera fixed in the car in front of the driver and the driver is alerted or warned if any of these actions of driver are detected

1.2 Fact & Statistics

The Road Safety Foundation said 7,713 people were killed and 12,615 injured in 6,829 road crashes last year. Last year's figures were the highest since 2019, it added. The number of road crashes and deaths in 2021 were 6,284 and 5,371. Of these, at least 20% were caused due to fatigue causing drivers to make mistakes. This can be a relatively smaller number still, as among the multiple causes that can lead to an accident, the involvement of fatigue as a cause is generally grossly underestimated. Fatigue combined with bad infrastructure in developing countries like Bangladesh is a recipe for disaster. Fatigue, in general, is very difficult to measure or observe unlike alcohol and drugs, which have clear key indicators and tests that are available easily. Probably, the best solutions to this problem are awareness about fatigue-related accidents and promoting drivers to admit fatigue when needed. The former is hard and much more expensive to achieve, and the latter is not possible without the former as driving for long hours is very lucrative.

1.3 Motivation:

motivation for developing (DMS) stems from several critical factors related to road safety, driver behavior, and technological advancements. Here are some key motivations for implementing a DMS project:

Enhancing Road Safety:

- One of the primary motivations for a DMS is to improve road safety by monitoring driver behavior and intervening when necessary to prevent accidents.

Reducing Accidents and Fatalities:

- Motor vehicle accidents are a leading cause of death worldwide. Implementing a DMS aims to reduce the number of accidents and fatalities by identifying risky driving behaviors and intervening before accidents occur

Minimize Accident Severity: Reduce the severity of accidents, as measured by the cost of repairs, medical expenses, and downtime, by [specify a measurable percentage] after the DMS is operational

Mitigating Risks in Autonomous Vehicles:

- As autonomous vehicle technology evolves, ensuring the safety of occupants and other road users remains a priority. DMS can play a crucial role in monitoring the driver's engagement and readiness to take control of the vehicle when necessary, especially in semi-autonomous or highly automated driving modes.

Addressing Driver Distraction and Fatigue:

- Driver distraction and fatigue are significant contributors to road accidents. With the proliferation of smartphones, in-vehicle entertainment systems, and other distractions, drivers may not always be fully focused on the road. A DMS can monitor driver attention levels and detect signs of drowsiness or distraction, allowing for timely interventions. **Enhance Safety Culture:** Foster a culture of safety within the organization, emphasizing the importance of responsible and alert driving practices among all drivers.

1.4 Objective

The main objective of a driver drowsiness detection system using Python and OpenCV is to monitor a driver's level of drowsiness and alert them when they are at risk of falling asleep or losing concentration while driving. This system aims to enhance road safety by detecting signs of drowsiness in real-time and providing timely warnings to prevent accidents caused by driver fatigue. And purpose of this project is to develop the simulation of drowsiness detection system. The focus of the project is to design a system that will detect the drowsiness by detecting the closed eyes of the driver. By monitoring the state of the eyes, it is believed can detect the early symptom of the driver's drowsiness, to avoid car accidents. The process of detecting the drowsiness between drivers is to detect the open and closed of the eyes

1.5 Techniques Found for this solution:

There are many techniques used to detect drowsiness of drivers; we will briefly discuss 3 of the main techniques.

The first technique is the driving pattern of the vehicle as monitoring steering wheel movement and sense if the driver is holding the steering wheel correctly using sensors so when driver doesn't have full control on the steering wheel it will lead to unusual driving pattern in addition to acceleration and braking time series which is done by tracking the acceleration and the brakes the user makes during his ride using sensors and any unusual change in acceleration or brakes series this means the driver is drowsy.

The second technique is that focus on electrical bio-signals such as EEG (Electroencephalography), ECG (Electrocardiography) and EOG (Electrooculogram).

Computer Vision techniques and deep learning for driver monitoring: this is done by detecting the driver's face using a camera and process the input using deep learning to output his drowsiness level based on yawning or blinking rate.

1.6 Advantages over other techniques:

each technique has its unique advantages and may be more suitable for specific applications or use cases within a Driver Monitoring System (DMS) project. The choice of technique depends on factors such as accuracy requirements, environmental conditions, user preferences, and integration complexity. Integrating multiple techniques may also offer synergistic benefits, providing a more comprehensive and robust solution for monitoring and managing driver behavior.

1.7 Target Audience:

The intended audience for the project is the development team, the project evaluation jury, and other enthusiasts who wish to further work on the project and who look forward for safely driving and less road accidents that helps the following customers:

Chauffeurs

Chauffeurs' workstyles oblige them to drive for long hours. Whether the chauffeur is a private chauffeur, cab driver, bus driver, truck driver, or even works in a company as Uber, all those examples are on risk of car accidents due to drowsiness.

The Elderly

Elder have higher risks of sudden heart attacks, or strokes. Detecting these risks, we will be able to take actions for saving their lives through SOS signals to emergency contacts and self

Cardiac Disease Patients

Patients with heart problems are in a risk to get heart attack while driving .
Detecting that we will be able to take actions regarding those risks.

Other Sudden Medical Emergencies

Everyone is in a risk even if they do not know it to get a stroke , heart attack , or even any other medical emergency. These cases should be handled by taking actions to make sure not to make an accident and send SOS signal.

Long Distance Drivers

All people with lifestyle that oblige them to drive for long hours whether their work is faraway or any other reason to drive for long hours. Those people are in a risk to sleep while driving

driving.

Chapter Two

2.1 Limitation of our System

Here are some of the limitation of our Project

- **Environmental Factors:**

DMS performance may be affected by environmental factors such as poor lighting conditions, glare, reflections, or adverse weather conditions. These factors can impact the accuracy of sensor readings and pose challenges for reliable detection of driver behavior.

- **User Acceptance:**

Some drivers may perceive DMS technology as intrusive or privacy-invasive, leading to resistance or reluctance to use it

- **Accuracy and Reliability:**

DMS may sometimes produce false positives or false negatives, inaccurately detecting driver distraction or drowsiness or failing to detect these states when they occur. This can lead to unnecessary alerts or missed opportunities for intervention.

- **Driver Compliance:**

Even if DMS alerts drivers to unsafe behaviors, there is no guarantee that they will respond appropriately or modify their behavior. Drivers may ignore or disable DMS alerts, override intervention systems, or engage in risky behavior despite warnings.

- **Complexity and Maintenance:**

DMS technology adds complexity to vehicle design, operation, and maintenance. It requires ongoing calibration, updates, and maintenance to ensure optimal performance and reliability over time. Failure to properly maintain DMS systems can compromise their effectiveness and safety.

2.2 Future Enhancement

When Detect Drowsy a few time's in 5 minute the system automatic control the

Car Engine and car switch off. System will be use in Mobile App and Web

Browser. can continue to evolve as integral components of vehicle safety systems, contributing to the vision of safer, more efficient, and more enjoyable mobility experiences for drivers and passengers alike.

2.3 Features of our System

- Real Time Face Detection
- Open Eyes Detection
- Close Eyes Detection
- Drowsiness Alert
- Database

2.4 Proposed System

The purpose of this project is to develop the simulation of drowsiness detection system. we present a plan for driver languor location. In this, the driver is ceaselessly observed through webcam. This model uses picture handling procedures which mostly centers on face and eyes of the driver The focus of the project is to design a system that will detect the drowsiness by detecting the closed eyes of the driver. By monitoring the state of the eyes, and detecting that driver is yawn By monitoring the state of the mouth so that we can detect the early symptom of the driver's drowsiness, to avoid car accidents.

2.5 Conclusion

The Driver Drowsiness Detection System aims to improve road safety by monitoring driver drowsiness in real-time. By leveraging Python, OpenCV, and adlib, the system detects signs of drowsiness, triggers timely alerts, and logs data for analysis. Although it has certain limitations, the proposed system holds significant potential for reducing accidents related to driver fatigue and contributing to safer driving experiences. The system will detect drowsiness by observing eye blinking patterns that is achieved by using Euclidean distance ratio eye blinking ratio

2.6 software Tools

- Python



- OpenCV



- Tensorflow

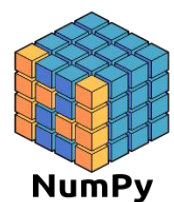


- sklearn



- pandas

- numpy



2.7 Hardware Tools

- C++



- Cmake



- **Camera:** the camera used for detecting the frames of the driver and send it to the board to make the processing . ex:Raspberry pi Camera



- **Screen:** To communicate with the driver behind the wheels



- **Speakers:** The driver is alerted in another way other than a message on the screen, which is a sound played on the speakers



- **SIM 808 GPS/GSM Module:** Since the driver could be already not responding to the alert messages on the screen, a fast action is needed to be taken. To minimize the damage and to prevent the loss of innocent lives, an SOS signal is sent to the emergency operator through the modem. The driver can make an emergency call.



- **USB to TTL Module:** he USB to TTL Module is a converter that is used to convert the serial ports into a single USB 2.0 type-A in order to be used by computers that do not have serial ports



Chapter Three

3.1 Single Board Computer Literature Review

For the board, we searched for a board that is an affordable one and contains a GPU. We found the NVIDIA Jetson Nano board and the Raspberry Pi 4 and we compared between them. Jetson Nano supports high-resolution sensors, can process many sensors in parallel and can run multiple modern neural networks on each sensor stream. It also supports many popular AI frameworks, making it easy for developers to integrate their preferred models and frameworks into the product. Jetson Nano joins the Jetson™ family lineup, which also includes the powerful Jetson AGX Xavier™ for fully autonomous machines and Jetson TX2 for AI at the edge. Ideal for enterprises, startups and researchers, the Jetson platform now extends its reach with Jetson Nano to 30 million makers, developers, inventors, and students globally. Jetson Nano Module: The design comes complete with power management, clocking, memory, and fully accessible input/outputs. Because the AI workloads are entirely software defined, companies can update performance and capabilities even after the system has been deployed.

Key features of Jetson Nano include:

- GPU: 128-core NVIDIA Maxwell™ architecture-based GPU
- CPU: Quad-core ARM® A57
- Video: 4K @ 30 fps (H.264/H.265) / 4K @ 60 fps (H.264/H.265) encode and decode
- Camera: MIPI CSI-2 DPHY lanes, 12x (Module) and 1x (Developer Kit)
- Memory: 4 GB 64-bit LPDDR4; 25.6 gigabytes/second
- Connectivity: Gigabit Ethernet
- OS Support: Linux for Tegra®
- Module Size: 70mm x 45mm
- Developer Kit Size: 100mm x 80mm

NVIDIA Jetson Nano is the latest board in NVIDIA IoT board lines. It is available as a Developer Kit that comprises all inputs and connections required for prototyping IoT solutions and, also, as a

compute module that can be integrated into industrial solutions. These developers kit delivers the compute performance necessary to run AI workloads at unprecedented power, cost, and size. Makers, learners, and developers can now run AI frameworks and models for applications, including object detection, speech processing, segmentation, and image classification. NVIDIA Jetson Nano vs. Raspberry Pi 4: They are both single board computers designed by ARM Processor and 4G of RAM and lots of connectivity for peripherals. However, the Raspberry Pi 4 also comes in an 8GB memory option. In terms of GPU, the Jetson Nano wins because of their 128- core Maxwell GPU @ 921 Mhz. The Raspberry Pi 4 GPU is weaker compared to the Jetson Nano. This CPU offers higher performance and faster clocking speed. But for deep learning and AI, it might not provide enough performance benefits. However, Raspberry Pi's faster performance makes it great for general-purpose use cases. For applications where higher GPU power is necessary or high-quality industrial product is required, NVIDIA Jetson Nano is the board to use. A NVIDIA's Jetson Nano has great GPU capabilities which makes it not only a popular choice for Machine Learning (ML), and it is also often used for gaming and CUDA based computations. Also, the benefit with the Nano is the access to the Jetson Package and a platform to learn and test the Cuda software. So, for our project we decided to use the Jetson Nano board as we need great GPU capabilities for our real time projec

3.2 Camera Literature Review

We started searching about cameras that are compatible with the Jetson Nano and we found different camera models like: USB camera, IMX219-77, IMX219-160IR, Camera raspberry pi CSI and below we are going to compare between them and choose the most suitable one.

USB camera:

Webcams generally have inferior quality to the camera modules that connect to the CSI interface. They can also not be controlled using the raspistill and rasivid commands in the terminal neither by the picamera recording package in Python. Nevertheless, there may be reasons why you want to connect a USB camera to your Raspberry Pi, such as because of the benefit that it is much easier to set up multiple cameras with a single Raspberry Pi. Webcams are unlikely to get the same performance as CSI, and they also use a LOT more CPU.

IMX219-77 Camera: The camera used for detecting the frames of the driver is called IMX219-77. This camera comes with the Developer kit of the NVIDIA Jetson Nano. It supports the Jetson Nano. It has an 8 Megapixels, has high resolution 3280 X 2464.

IMX219-160IR Camera: An infrared camera detects and measures the infrared energy of objects. The camera converts that infrared data into an electronic image that shows the apparent surface temperature of the object being measured.

IMX219-160 IR features :

- Supports CM3/3+/4, Jetson Nano, Jetson Xavier NX
- Supports night vision when works with infrared LEDs
- Megapixels
- Sensor: IMX219
- Resolution: 3280 × 2464
- 4 screw holes: Used for attachment, provides 3.3V

power output, Supports connecting infrared LED and/or

fill flash LED

- Dimension: 25mm × 24mm

It is not easily available to buy in Egypt and its output is not suitable to our project as our dataset is different than the output of this infrared camera.

Camera raspberry pi CSI: RPi Cams uses the special MIPI CSI camera format, designed for smartphone cameras to use less power, smaller physical size, faster bandwidth, higher resolutions, higher framerates, and reduced latency, compared to USB. the RPi Cams can often deliver 1920×1080 30 FPS or even Ultra HD / 4K resolutions. The RPi Cam v1 can in fact deliver up to 2592×1944 15 FPS or 1920×1080 30 FPS video even on a \$5 Raspberry Pi Zero, thanks to the use of MIPI CSI for the camera and a compatible video processing ISP & GPU hardware inside the Raspberry Pi. The RPi Cam also supports 640×480 in 90 FPS mode (such as for slow-motion capture), and this is quite useful for real-time computer vision so you can see very small movements in each frame, rather than large movements that are harder to analyze.

3.3 GPS/GSM module Literature Review

The first GPS/GSM module that can be used for getting the location, sending messages to and calling SOS contacts in case of emergencies is SIM808. In addition to sending the driver's location to the SOS contacts for rescue. So, we used the SIM808 GPS/GSM module as it has many benefits. It features ultra-low power consumption in sleep mode and integrated with charging circuit for Li-Ion batteries, that make it get a super long standby time and convenient for projects that use rechargeable Li-Ion battery. It has high GPS receive sensitivity with 22 tracking and 66 acquisition receiver channels. Besides, it also supports A-GPS that available for indoor localization. The module is controlled by AT command via UART and supports 3.3V and 5V logical level. The package includes 1 x GSM GPRS SIM808 Module SMS Chip Development Board.

Sample Applications of SIM808:

- Send a text message in English.
- Specified calls, automatically hang up after 1 minute.
- Received specified text messages, control LED open circuit.
- Detection module registered to the network, send the message until reading a SIM card.
- Connect to the specified server and send the specified data by GPRS.
- Provide information of dormant power consumption at about 10 MA.

Features of SIM808:

- Quad-band 850/900/1800/1900MHz.
- GPRS multi-slot class12 connectivity: max. 85.6kbps(down-load/up-load).
- GPRS mobile station class B.
- Controlled by AT Command (3GPP TS 27.007, 27.005 and SIMCOM enhanced AT Commands).
- Supports charging control for Li-Ion battery.
- Supports Real-Time Clock.

- Integrated GPS/CNSS and supports A-GPS.
- Low power consumption, 10mA in sleep mode.
- Supports GPS NMEA protocol.
- AT command operation, can support SMS, phone, GPRS data, GPS data is automatically output, embedded TCP protocol to support transparent mode and command mode, supports HTTP protocol to support DTMF decoding, support MMS, support recording function, etc.
- Standard SIM Card.

When compared to **SIM908 modules**, **SIM808 GSM/GPS** modules are superior since they include SSL security for data transmission. GNSS-capable navigation system that is more sensitive (sensitivity of -165 dBm), as opposed to SIM908's GPS-only sensitivity of -160 dBm. When a device enables navigation utilizing both Russian and American satellites (referred to as GPS and GLONASS, respectively), the name GNSS is used (called GLONASS).

Compared to the SIM908, which comes in a package with dimensions of 30*30*3.2mm, the SIM808 module is much smaller, measuring only 24*24*2.6 mm. Therefore, the new SIM808 module should be used when constructing a new system as it is superior to SIM908 in every way. It is sturdier, smaller, and better at tracking. If you must adhere to strict size restrictions, you should utilize the SIM808 module. The SIM808 is roughly half a centimeter less in size than the SIM908 module, which results in significant PCB real estate savings. Due to their compatibility with all four 2G GSM bands used globally, SIM908 and SIM808 can both be used internationally. For the project we tried to buy SIM7600G-H 4G / 3G / 2G / GNSS Module for Jetson Nano, as this is a 4G/3G/2G communication and GNSS positioning module designed for Jetson Nano, it supports global LTE CAT4 up to 150Mbps for downlink data transfer, with pretty low power consumption. You can just attach it onto the Jetson Nano Developer Kit, easily enable functions like 4G high speed connection, wireless communication, remote video monitoring, making telephone call, sending SMS, global positioning, and so on, but it was really expensive and not available to be shipped to Egypt

Chapter Four

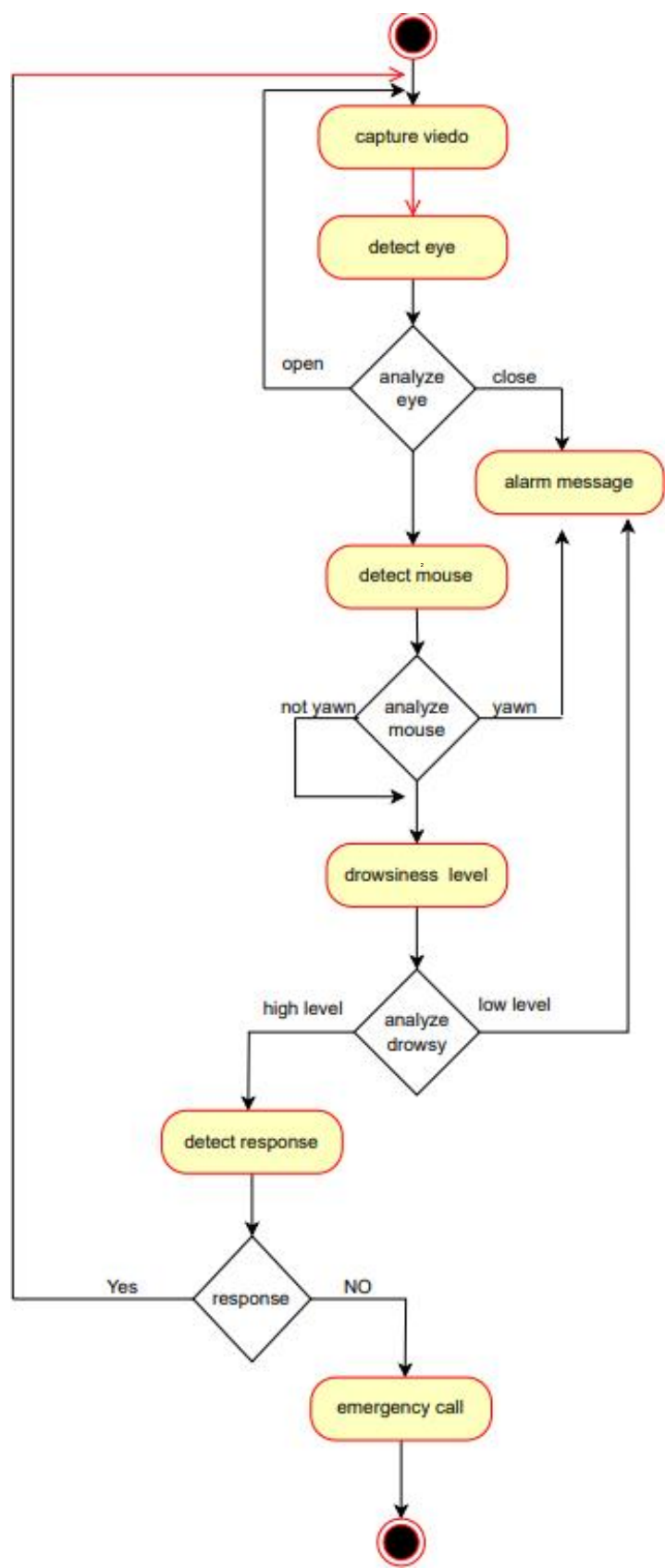
4.0 Project Scope

As mentioned before, there are many products out there that provide the detection of drivers' drowsiness which are implemented in many vehicles. This driver drowsiness detection system provides the functionalities that may seem similar to the previous work, with better results and additional features. The system aims to not only detect the driver's drowsiness but also measure the driver's drowsiness level. In addition, the system has a specific action to make with respect to the drowsiness level of the driver with a suitable alert.

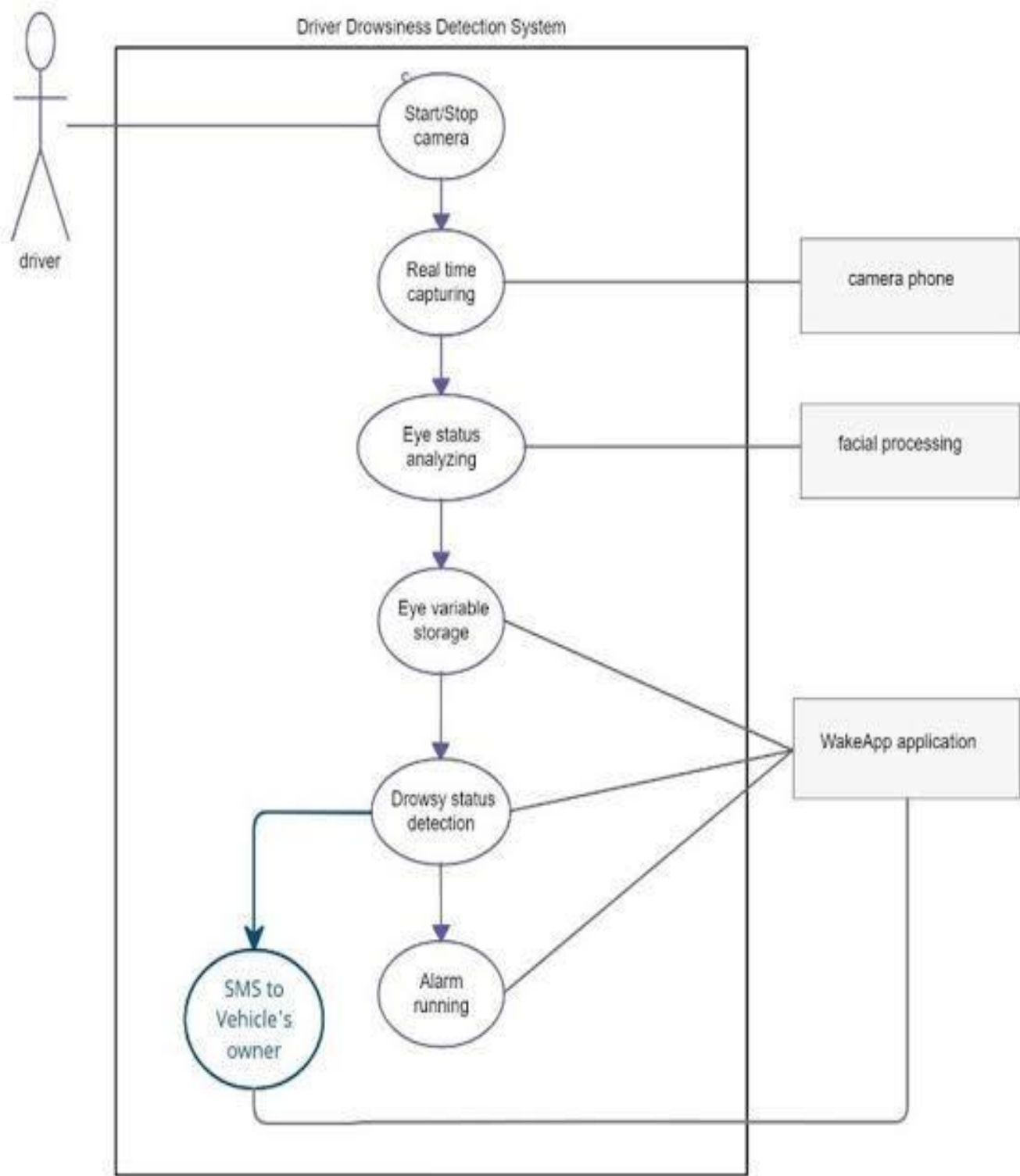
4.1 Features

- System detects the drowsiness level of the driver
- Specific action is taken according to the driver's drowsiness level
- Actions taken are accompanied with a suitable alarm system with respect to the drowsiness level
- Emergency message is sent with location in case of high-risk drowsiness to emergency contacts and ambulance
- If the vehicle's speed exceeds 120 km/h, the system sends an alert to the driver to slow down. The alarm continues to work until the speed is lower than 120 km/h.
- The driver can make emergency call to the emergency contact saved in the system

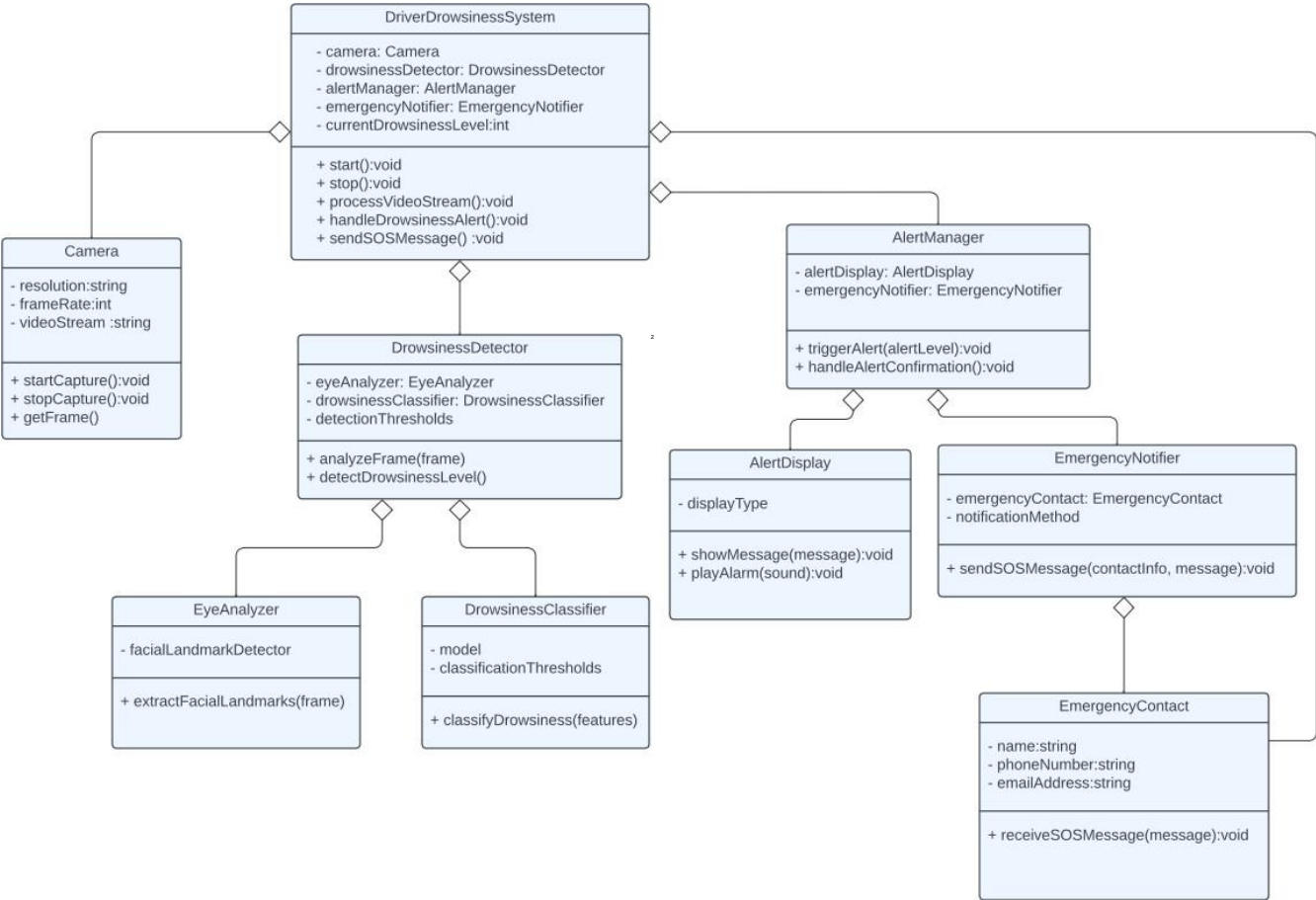
4.2 Activity Diagram



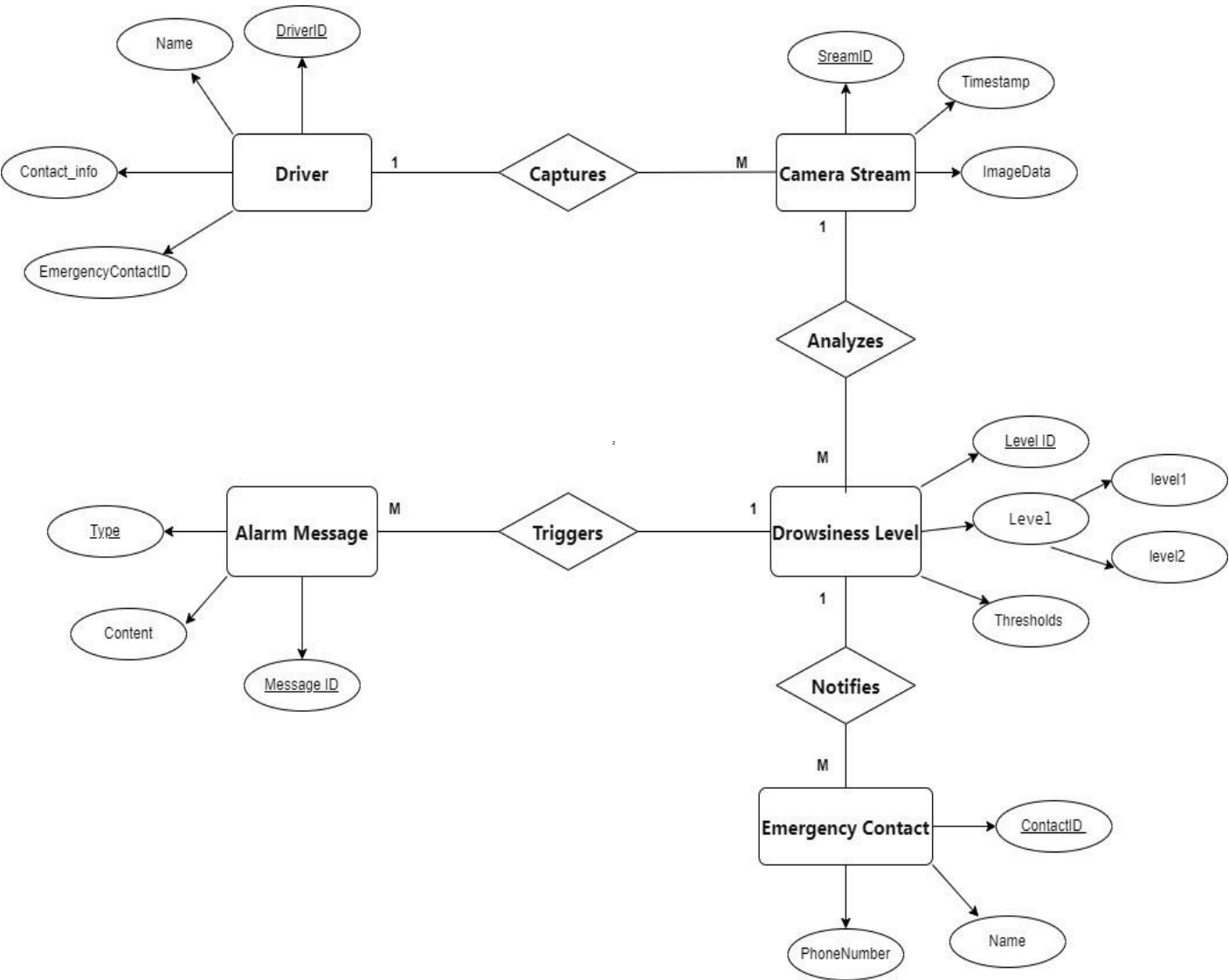
4.3 use case



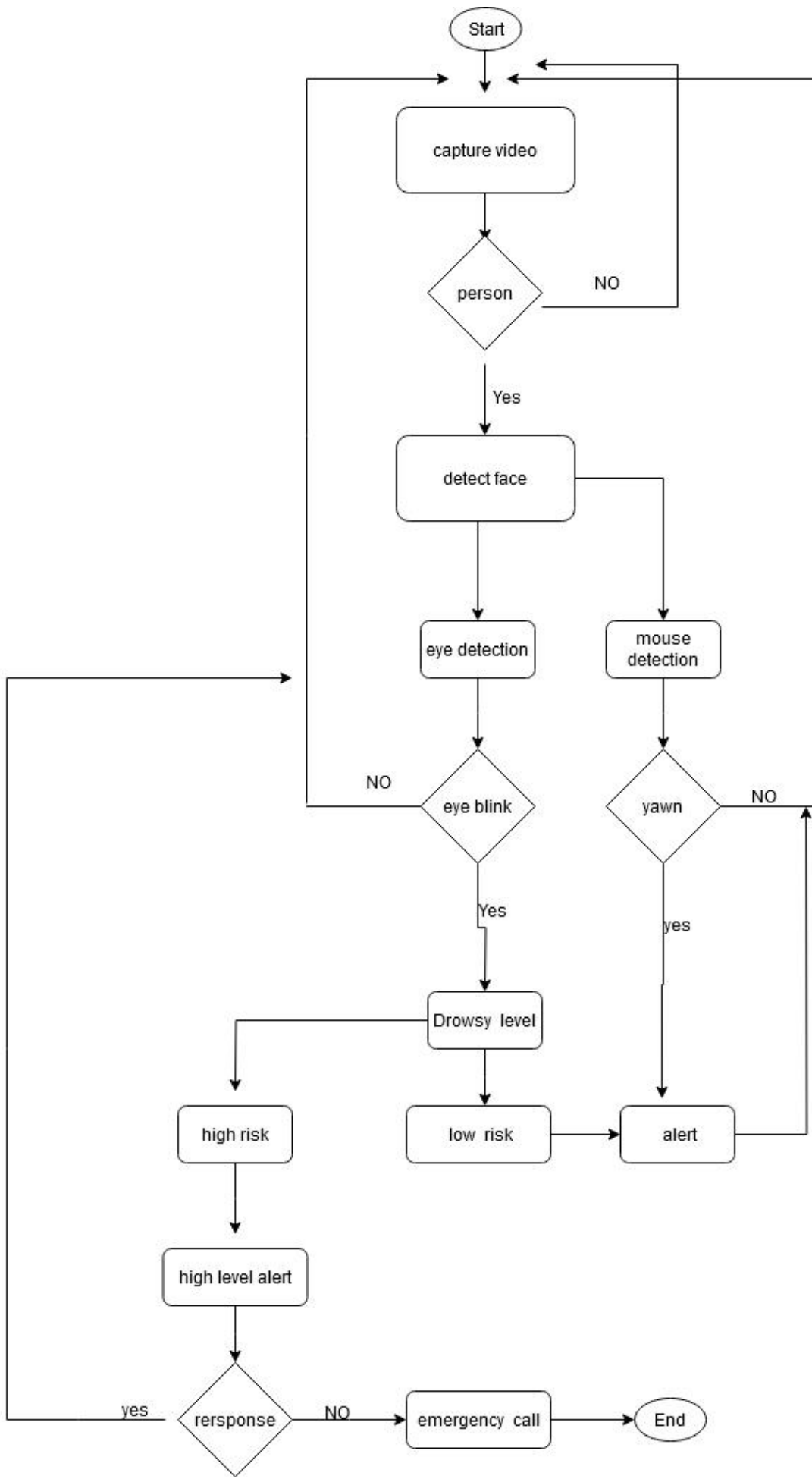
4.4 Class Diagram



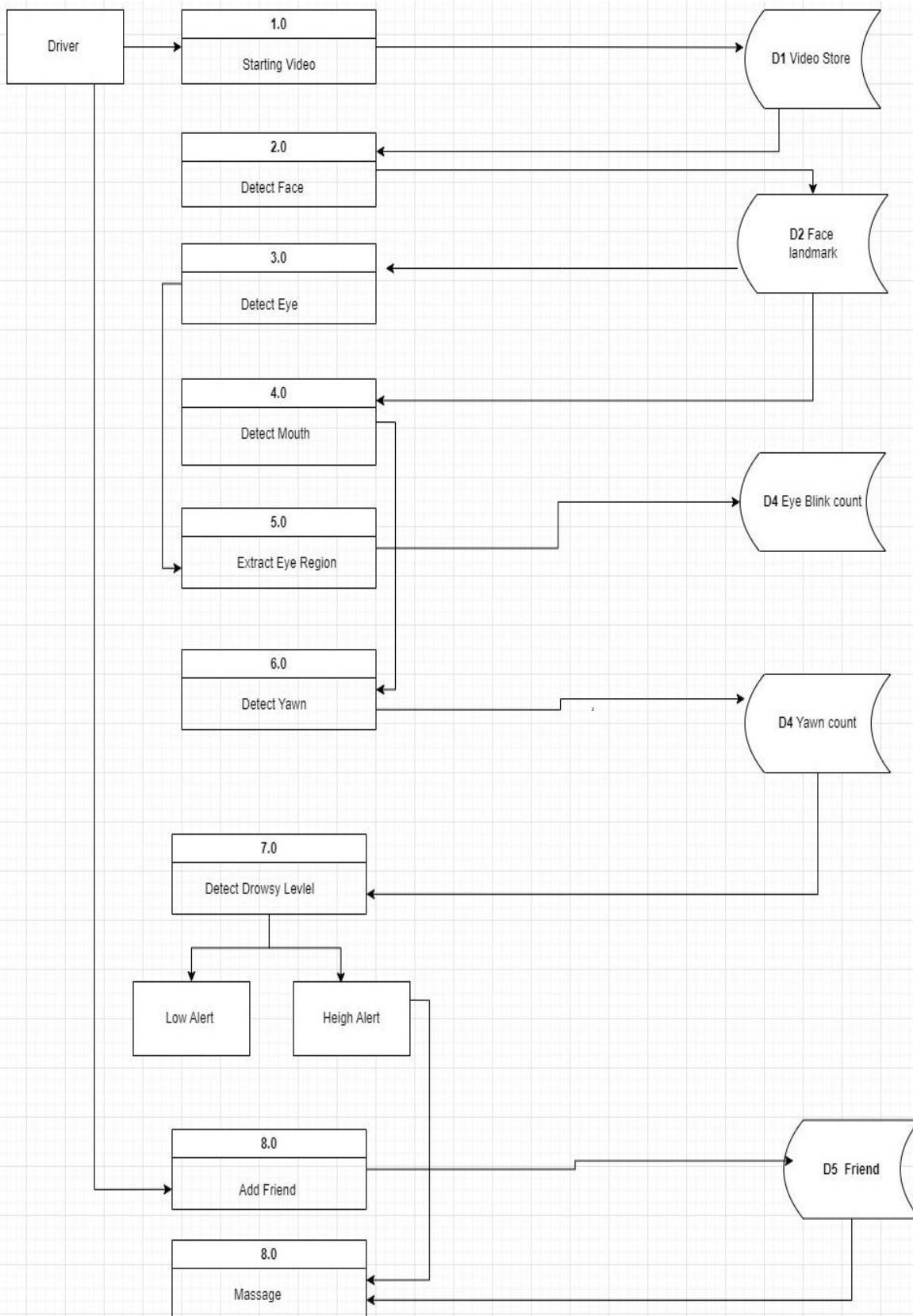
4.5 ERD



4.6 Flow chart



4.7 Data Flow Diagram



Chapter Five

5.1 open or close eye:

We obtained the dataset from Kaggle, and the dataset is divided into two types: the first type is the eyes, and the second is the mouth. The first type is divided into closed and open, while the second type is whether it's yawning or not yawning. After obtaining the dataset, we created models: one model predicts the state of the eyes, and the other predicts the state of the mouth. These models determine whether the driver is drowsy or not. We used Convolution neural network to build these models from scratch .building a Convolutional Neural Network (CNN) using TensorFlow and Keras to classify eye images as "Open" or "Closed." It does the following:

1. Data Processing: Loads and preprocesses images, then splits them into training and validation sets.
2. Data Augmentation: Applies various transformations to augment the training dataset, enhancing model robustness.
3. Model Architecture: Constructs a CNN with convolutional and pooling layers followed by fully connected layers. Dropout layers are used for regularization, and the final layer employs softmax activation for classification.
4. Model Training: Compiles and trains the model using the Adam optimizer and sparse categorical cross-entropy loss function. Data augmentation is applied during training.
5. Model Evaluation: Evaluates the model on the validation set, computing loss=0.0072, accuracy=0.996, and generating a classification report.
6. Model Saving and Loading: Saves the trained model to a file and reloads it for future use.
7. Testing and Visualization: Applies the model to test images, predicts labels, and visualizes results using Matplotlib.

And inference it in realtime stream:

Necessary Libraries:

Utilizes OpenCV, NumPy, warnings, and time for image processing and model inference.

Initialize Detectors and Model:

Creates instances of face, left eye, and right eye detectors, along with a drowsiness prediction model.

Loads pre-trained models and cascade files for accurate detection.

Initialize Variables:

Sets up counters for frames, drowsiness scores, and variables for rectangle thickness and previous eye state predictions.

Open Video Capture Device:

Initiates the default camera using webcam .

Record Start Time:

Logs the start time for calculating Frames Per Second (FPS).Captures and processes video frames .

Read Frame and Convert to Grayscale:

Reads frames from the camera and converts them to grayscale for face and eye detection.

Detect Faces, Left Eyes, and Right Eyes:

Utilizes Haarcascade classifiers to detect faces, left eyes, and right eyes. Draws rectangles around detected faces.

Process Left Eye:

Extracts the left eye region, predicts its state using a neural network model, and displays predictions on the frame.

Process Right Eye:

Similar to the left eye, processes the right eye and displays predictions on the frame.

Display Eye States:

Presents the predicted states of the left and right eyes on the frame.

Update Drowsiness Score:

Updates the drowsiness score based on eye state predictions and displays it on the frame.

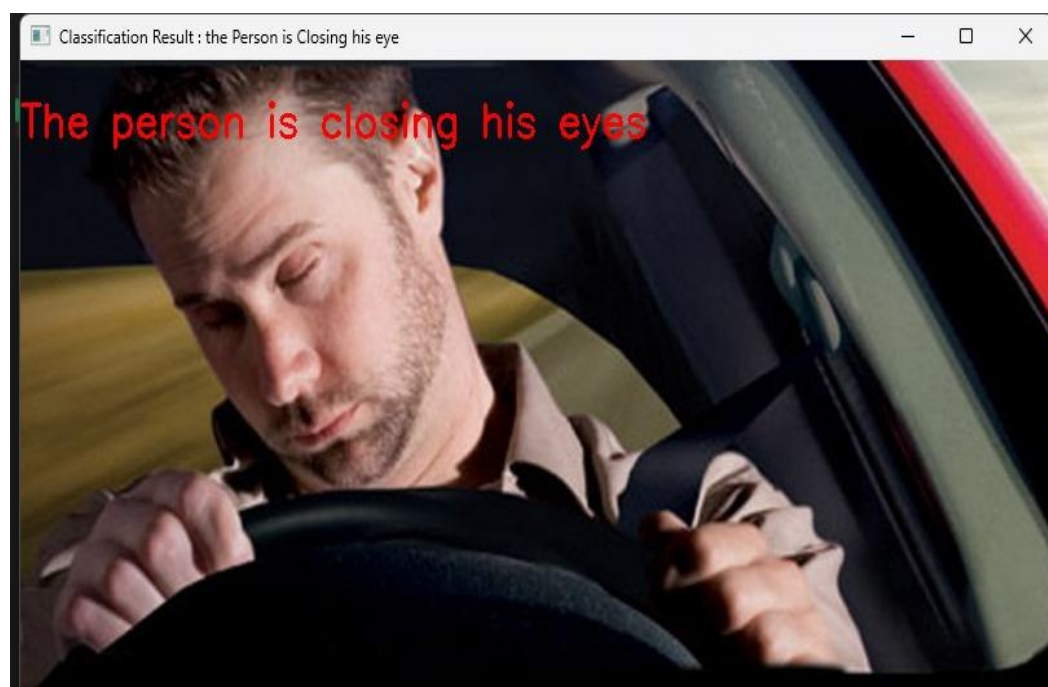
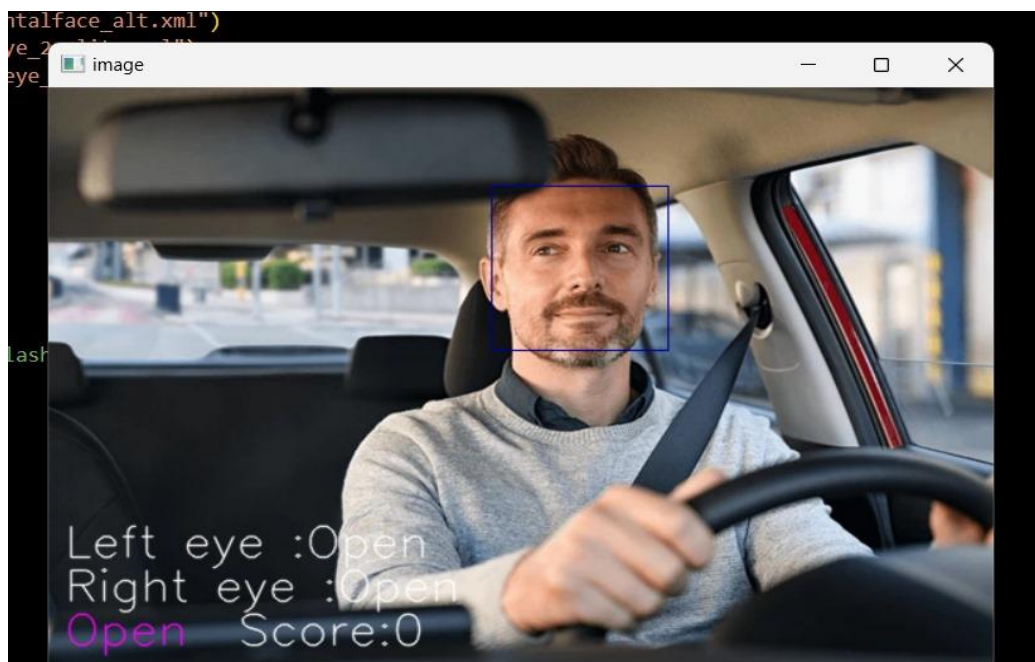
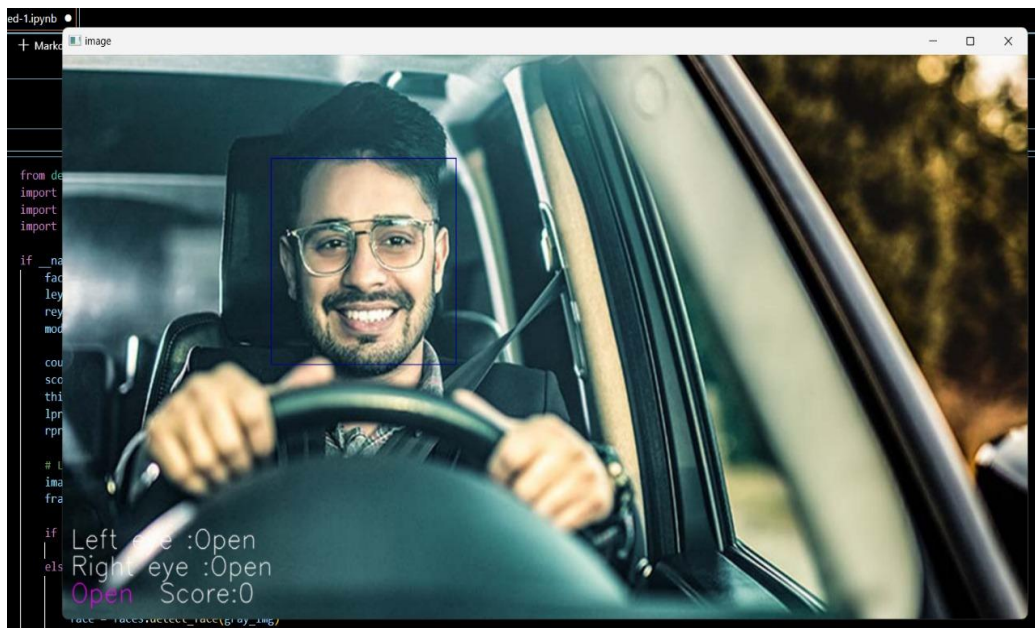
Display FPS:

Calculates and displays Frames Per Second (FPS) for performance monitoring.

Issue Warning and Draw Rectangle for Drowsiness:

Warns and draws a rectangle around the frame if the drowsiness score exceeds a specified threshold. Adjusts rectangle thickness based on the score.

Display Final Frame:Shows the final frame with eye states, drowsiness score, and FPS.



5.2 yawn or not yawn :

These models determine whether the driver is drowsy or not. We used Convolution neural network to build these models from scratch . building a Convolutional Neural Network (CNN) using TensorFlow and Keras to classify mouth images as "yawn" or "not yawn." It does the following:

1. Data Processing: Loads and preprocesses images, then splits them into training and validation sets.
2. Data Augmentation: Applies various transformations to augment the training dataset, enhancing model robustness.
3. Model Architecture: Constructs a CNN with convolutional and pooling layers followed by fully connected layers. Dropout layers are used for regularization, and the final layer employs softmax activation for classification.
4. Model Training: Compiles and trains the model using the Adam optimizer and sparse categorical cross-entropy loss function. Data augmentation is applied during training.
5. Model Evaluation: Evaluates the model on the validation set, computing loss=0.3001, accuracy=0.8684, and generating a classification report.
6. Model Saving and Loading: Saves the trained model to a file and reloads it for future use.
7. Testing and Visualization: Applies the model to test images, predicts labels, and visualizes results using Matplotlib



And inference_with tensorflow:

Necessary Libraries:

matplotlib, NumPy, tensorflow, and time for image processing and model .

Load the pre-trained TensorFlow model.

Set the target input dimensions of the model.

Load and preprocess an image for inference to . Ensure that the input data is preprocessed in the same way it was during training.

Used the loaded model to make predictions on the preprocessed input data by using threshold, if threshold equal to zero thin its Yawning detected else Not yawning

```
1/1 [=====] - 0s 246ms/step
Not yawning.
```



inference_with Tflite:

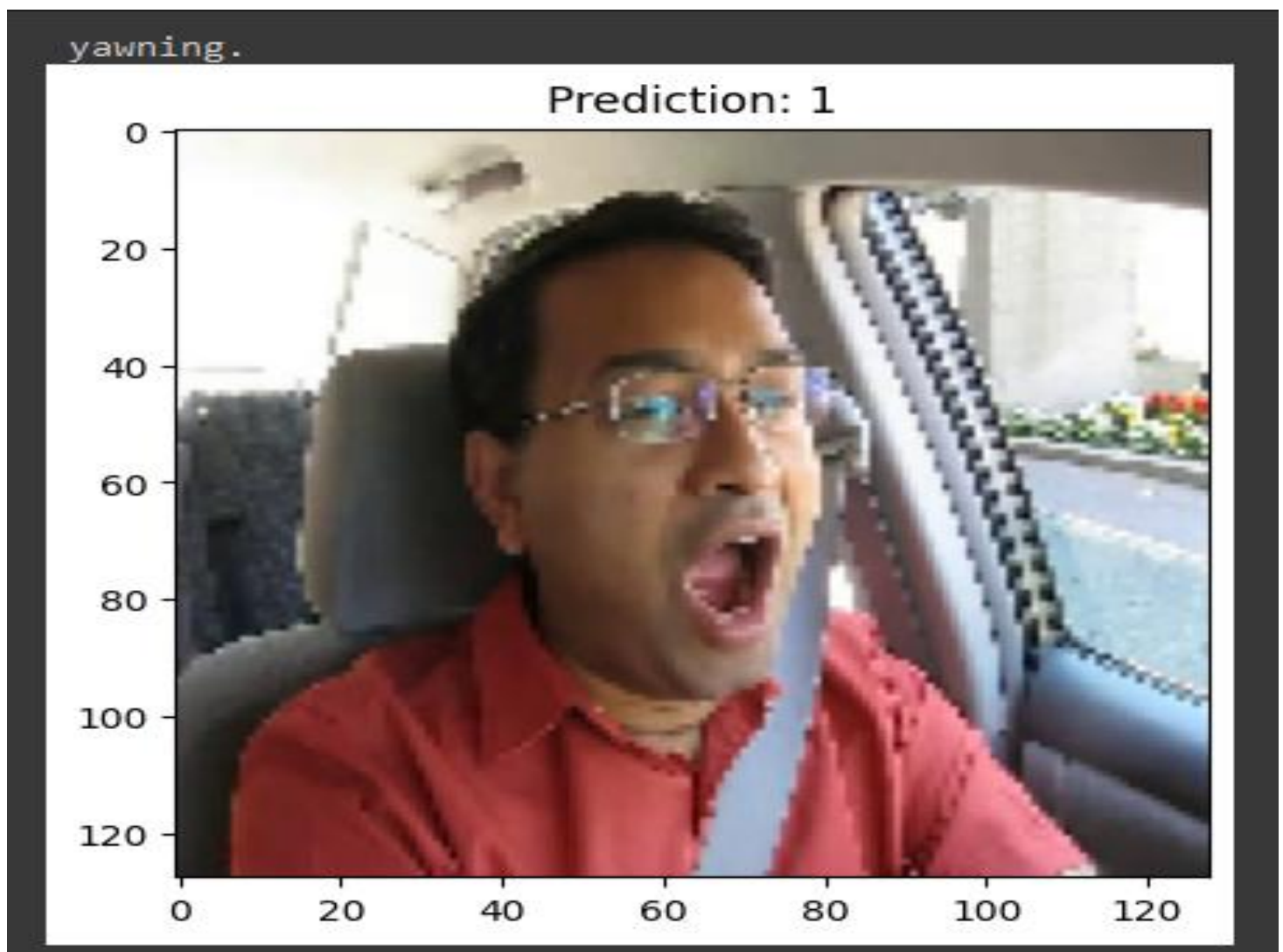
Load the TFLite Model:

Load and preprocess an image for inference

Get input and output details

Set input tensor

Interpret results by using threshold , if threshold bigger than 0.5 then its Yawning detected else Not yawning



5.3 mediapipe_eye state

Attributes:

- **detector:** Instance of `FaceMeshDetector` initialized with `maxFaces=1`.
- **angle_degrees:** Default angle value set to 45 degrees.
- **open:** Constant representing the state of an open eye (value: 0).
- **closed:** Constant representing the state of a closed eye (value: 1).
- **left_eye_state:** Current state of the left eye, initialized as open.
- **right_eye_state:** Current state of the right eye, initialized as open.
- **right_eye_id_list:** List of key points indices for the right eye.
- **right_eye_normalizing_lst:** List of key points indices used for normalizing distances for the right eye.
- **left_eye_id_list:** List of key points indices for the left eye.
- **left_eye_normalizing_lst:** List of key points indices used for normalizing distances for the left eye.
- **triangle_points:** List of indices defining key points of a triangle for angle calculation.

Methods:

1. **detect_V2H_per_left_eye(detected_face, normalizing_points_idx_lst):**
 - Calculates vertical-to-horizontal (V2H) ratio for the left eye based on detected face and normalizing points.
2. **detect_V2H_per_right_eye(detected_face, normalizing_points_idx_lst):**
 - Calculates vertical-to-horizontal (V2H) ratio for the right eye based on detected face and normalizing points.
3. **fraction(detected_face, triangle_points):**
 - Computes the angle between two vectors formed by key points of a triangle on the face.
4. **detect_left_eye_state(V2H_ratio_left, previous_state, thresh_from_open_to_closed, thresh_from_closed_to_open):**
 - Determines the state of the left eye (open or closed) based on V2H ratio and previous state.

5. **detect_right_eye_state(V2H_ratio_right, previous_state, thresh_from_open_to_closed, thresh_from_closed_to_open):**

- Determines the state of the right eye (open or closed) based on V2H ratio and previous state.

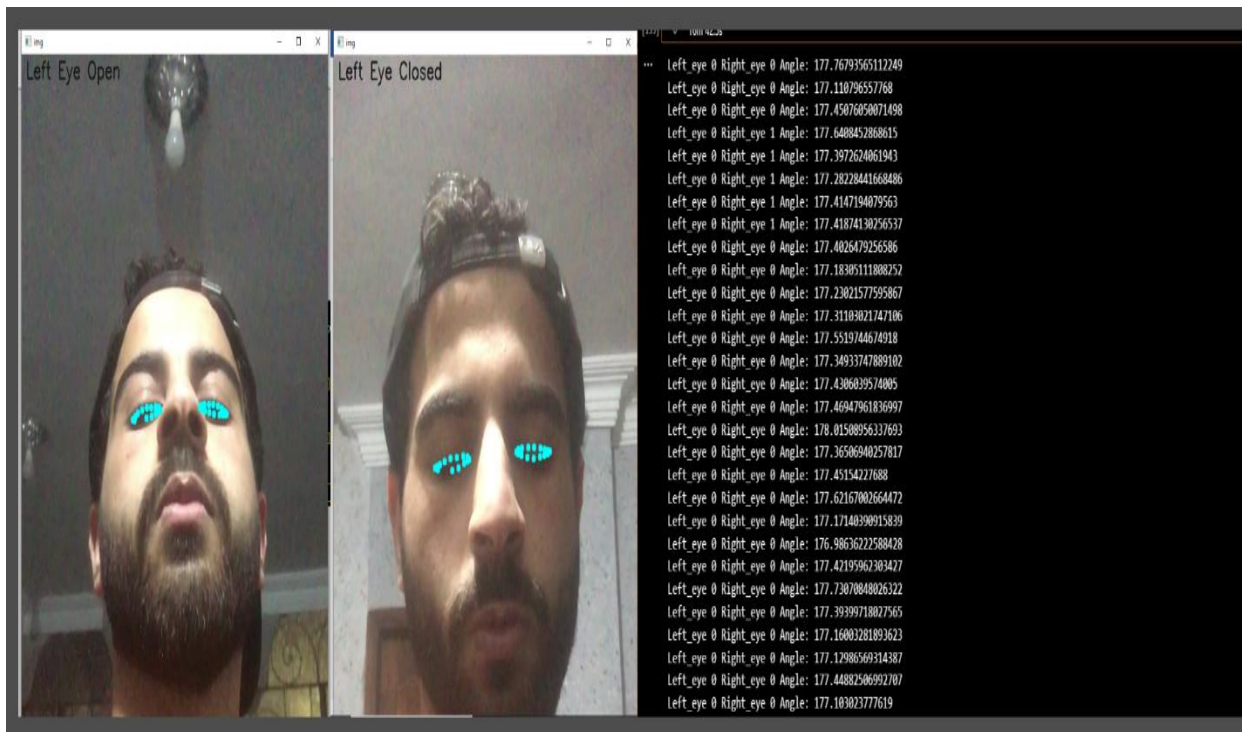
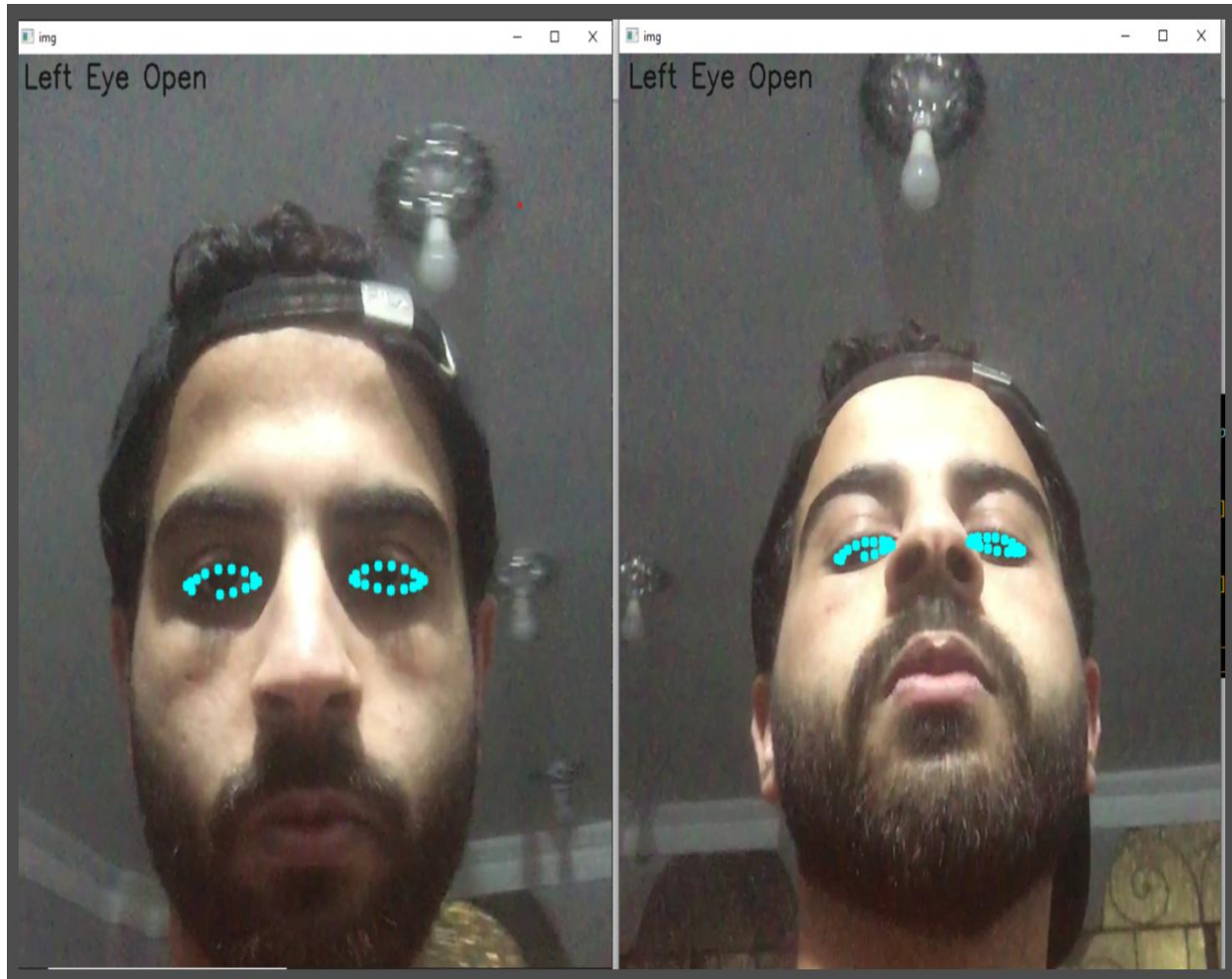
6. **visualise(img, detected_face):**

- Visualizes key points for both eyes and triangle points on the image.
- Displays the state of the left eye on the image.

7. **detect(img, visualise=False):**

- Detects features on the provided image.
- Optionally visualizes the detected features if **visualise** parameter is set to True.
- Returns the modified image with detected features, states of left and right eyes, and detected face.

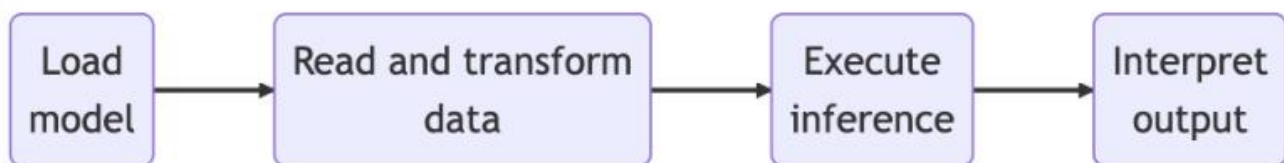
Output:



5.4 Deploying models on our Embedded Target

The Framework that we used to deploy model is TensorFlow lite Using TensorFlow Lite C++ API

- **TensorFlow lite** : TensorFlow Lite is a deep learning framework for local inference, specifically for the low computational hardware. It allows on-device machine learning by assisting developers in running their models on compatible hardware and IoT devices.
- TensorFlow Lite inference typically follows the following steps:



Loading a model You must load the .tflite model into memory, which contains the model's execution graph. This model is stored in the FlatBufferModel class, and you can create an instance of it using the BuildFromFile method with the model file name as the input argument.

```
auto model = tflite::FlatBufferModel::BuildFromFile(model_file.c_str());

if (!model) {
    throw std::runtime_error("Failed to load TFLite model");
}
```

Next, we build an Interpreter—the class that will take the model and execute the operations it defines on input data while also providing access to the output. To do so, we use the [InterpreterBuilder](#), which will allocate memory for the Interpreter and manage the setup so that the Interpreter can read the provided model. Note that before running the inference, we tell the interpreter to allocate memory for the model's tensors by calling the [AllocateTensors\(\)](#) method. In the last line of this block, we also call [PrintInterpreterState](#), a debugging utility useful for inspecting the state of the interpreter nodes and tensors.

```

tflite::ops::builtin::BuiltinOpResolver op_resolver;

std::unique_ptr<tflite::Interpreter> interpreter;

tflite::InterpreterBuilder(*model, op_resolver>(&interpreter);

if (interpreter->AllocateTensors() != kTfLiteOk) {
    throw std::runtime_error("Failed to allocate tensors");
}

tflite::PrintInterpreterState(interpreter.get());

```

● Read and transform the input data

Now, with our interpreter prepared to receive data and perform the inference, we must first adapt our data to match the input format accepted by the model. In this section, we'll outline the following process:

Read the input video .

Crop the input video frame to create a square image . Resize the image to match the input accepted by the model .

```

auto input = interpreter->inputs()[0];

auto input_batch_size = interpreter->tensor(input)->dims->data[0];
auto input_height = interpreter->tensor(input)->dims->data[1];
auto input_width = interpreter->tensor(input)->dims->data[2];
auto input_channels = interpreter->tensor(input)->dims->data[3];

std::cout << "The input tensor has the following dimensions: ["
    << input_batch_size << ","
    << input_height << ","
    << input_width << ","
    << input_channels << "]" << std::endl;

```


- **Running inference**

After preparing and copying the input data to the input tensor, we can finally run the inference. This can be done by calling the `Invoke()` method of the interpreter. If the inference runs successfully, we can recover the output tensor from the model by getting `typed_output_tensor` from the interpreter..

```
if (interpreter->Invoke() != kTfLiteOk) {  
    std::cerr << "Inference failed" << std::endl;  
    return -1;  
}  
  
float *results = interpreter->typed_output_tensor<float>(0);
```

- **Interpreting output** :When you receive results from the model inference, you must interpret the tensors in a meaningful way that's useful in your application. For example, a model might return only a list of probabilities. It's up to you to map the probabilities to relevant categories and present it to your end-user
- **Build the project** To build this project, we utilize CMake, a powerful and crossplatform build system.



